

Design Documentation

Agentic RAG-Based Citizen Service Advisory Chatbot

GitHub Repository: github.com/abhi6744/Agentic-RAG-Based-Citizen-Service-Advisory-Chatbot

Submitted by Abhinav G

Roll No: 25225040

Under the guidance of

Dr. Ramesh Chandra Poonia

Dr. Shilpa Shrivastava



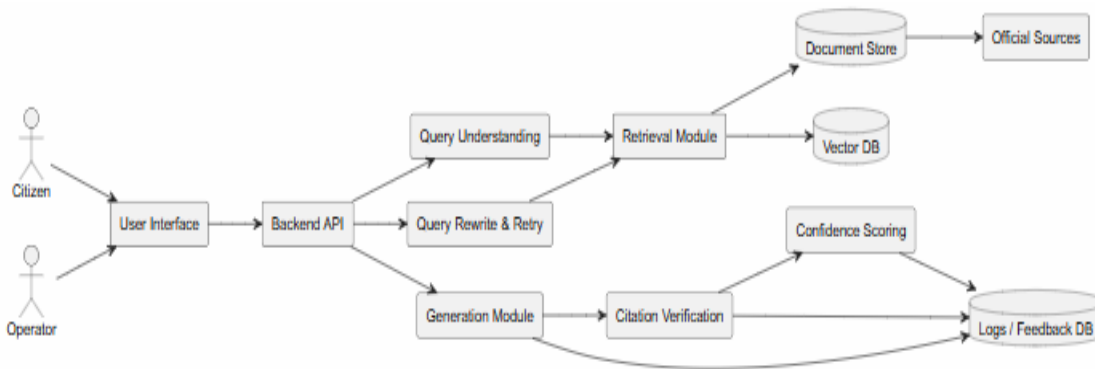
CHRIST
(DEEMED TO BE UNIVERSITY)
BANGALORE | DELHI NCR | PUNE

School of Sciences

2026-2027

1. System Architecture

The system is organized into a user interface, backend API, retrieval layer, generation layer, citation verification layer, and logging layer. The architecture is designed so that the model does not answer from memory alone; it first retrieves trusted sources and then generates a grounded response. This keeps the system suitable for advisory use in citizen service centres where accuracy matters more than speed alone.



2. Module Design

2.1 Query Understanding Module

This module receives the user query, detects intent, and identifies whether the user is asking about service selection, eligibility, required documents, identity mismatch, or next-step guidance. It also normalizes the text and prepares it for retrieval. If the query is in mixed English and Malayalam, the module should preserve meaning while cleaning the input.

2.2 Retrieval Module

This module converts the query into embeddings and searches the knowledge base for relevant service documents, FAQs, and official guidance pages. It should use top-k retrieval and optional metadata filtering so that only service-related evidence is returned. Retrieval quality is critical because the generated answer should come from the most relevant sources.

2.3 Query Rewrite and Retry Module

If the first retrieval attempt is weak, this module reformulates the query and runs retrieval again. This is useful for vague, incomplete, or badly phrased citizen queries. The goal is to improve the chance of finding usable official information before the system decides whether confidence is sufficient.

2.4 Generation Module

This module uses the retrieved context to create a clear, structured answer. The response should explain eligibility, documents required, fees, and next steps in simple language. The answer should remain concise but complete enough for a citizen to act on.

2.5 Citation Verification Module

This module checks whether the generated answer is supported by retrieved sources. If a claim cannot be linked to evidence, the answer should either be rewritten or marked as low confidence. This module is important because the project is designed to be source-grounded rather than purely generative.

2.6 Confidence Scoring Module

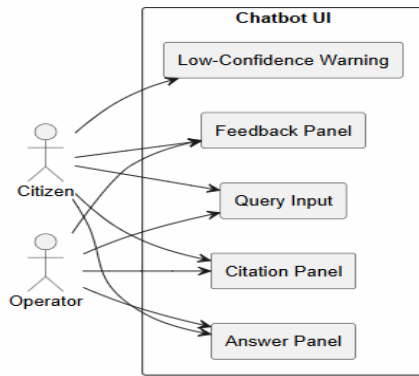
This module evaluates how strong the retrieved evidence is and whether the answer can be trusted. If the score is below a threshold, the system should warn the user and recommend verification at the centre. The confidence score can be based on similarity, source quality, and evidence coverage.

2.7 Logging and Feedback Module

This module stores query text, response text, citations, confidence score, and user feedback. These logs are useful for evaluation, debugging, and future improvement. They also help track which types of questions are asked most often.

3. UI Design — Citizen and Operator View

The user interface should be simple and accessible. Citizen mode should allow a user to ask a question and read the answer with citations. Operator mode should support structured summaries, common issue categories, and quick reference to related documents. The interface should make the system useful in a practical service-centre environment.

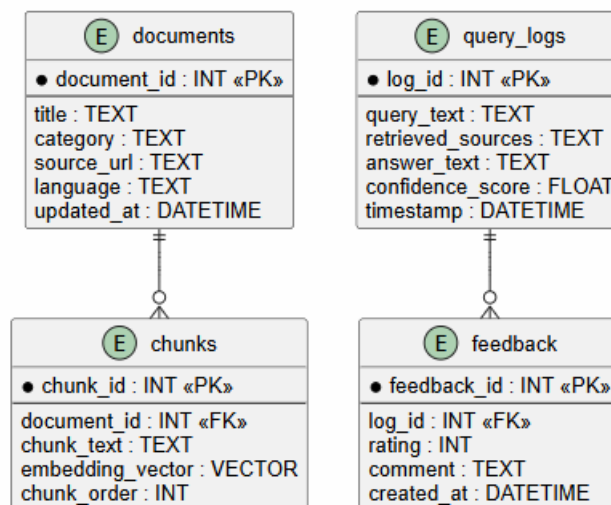


3.1 Panel Descriptions

- Query Input: accepts citizen questions in English or Malayalam.
- Answer Panel: shows the generated response with clear sections.
- Citation Panel: lists the supporting source references.
- Low-Confidence Warning: alerts when evidence is weak or incomplete.
- Feedback Panel: allows the user or operator to rate the answer.

4. Database Design

The system should store document metadata, chunk embeddings, query logs, and feedback in a lightweight relational database and/or vector database depending on the implementation stack. Document records should include title, category, source, date, and language. Chunk records should connect each text block to its parent document so that citations can be traced correctly. Query logs should store the query text, confidence score, and response for evaluation only, preferably in anonymized form.



4.1 Table: documents

This table stores the metadata for official service documents and guidance pages. Fields may include document_id, title, category, source_url, language, and updated_at. It is the main entry point for traceability because each chunk should point back to one document record.

4.2 Table: chunks

This table stores split text blocks and their embeddings. Each chunk should include chunk_id, document_id, chunk_text, embedding_vector, and chunk_order. The table supports semantic search and citation mapping during answer generation.

4.3 Table: query_logs

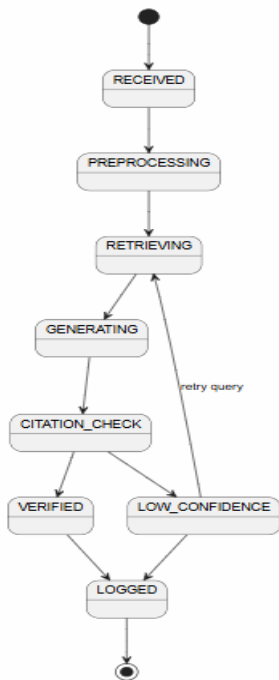
This table stores query text, retrieved sources, answer text, confidence score, and timestamp. It helps evaluate the quality of retrieval and generation. If privacy is a concern, user-specific data should be avoided or anonymized.

4.4 Table: feedback

This table stores the rating and optional comment provided by the user or operator. Feedback helps identify weak responses and improve the knowledge base. It can also be used to fine-tune future retrieval rules.

5. Query Lifecycle (State Design)

A query begins when a citizen submits it, moves through preprocessing, retrieval, generation, and citation checking, and ends either in a final answer or a low-confidence warning. The state model is useful because it shows how the system behaves when evidence is strong versus when the system needs to retry or warn. This makes the response pipeline easier to test and debug.



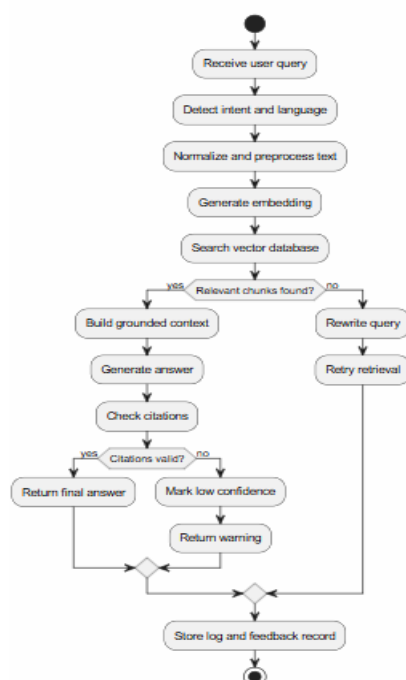
5.1 State Semantics

RECEIVED means the query has entered the system. PREPROCESSING means the text is being cleaned and interpreted. RETRIEVING means the system is searching documents. GENERATING means the answer is being composed. VERIFIED means citations are checked and the answer is supported. LOW_CONFIDENCE means evidence is weak and the user should be warned. LOGGED means the request and response have been stored.

6. Algorithm Design

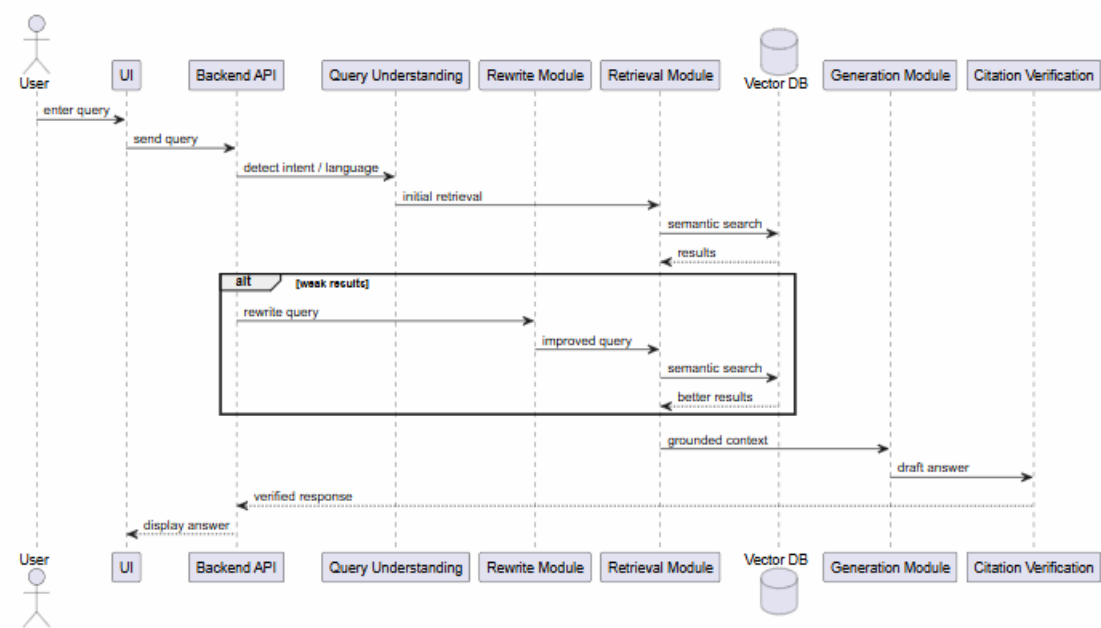
6.1 Retrieval and Generation Algorithm

Every query first passes through preprocessing and intent detection. The system then searches the knowledge base, assembles the most relevant context, generates the answer, and checks citations before returning the response. If the evidence is insufficient, it retries or returns a warning. This keeps the chatbot grounded and practical for official service guidance.



6.2 Query Rewrite Algorithm

If the initial query is too vague, the system creates a clearer version of it and performs another search. This helps when a user asks something like “documents for name change” without specifying the exact service. The rewrite step improves retrieval without forcing the user to retype the query manually.



6.3 Citation Validation Algorithm

The system compares each generated claim with the retrieved text chunks. If a claim has no support, the claim should either be removed, rewritten, or marked as uncertain. This ensures that the system does not present unsupported advice as fact.

7. Traceability Note

This design document maps directly to the SRS requirements for query understanding, multilingual support, retrieval, citation-grounded answers, confidence scoring, logging, and feedback. The design sections explain how those requirements are implemented in modules and workflows. The result is a practical blueprint for building the advisory chatbot.